PRUEBA TÉCNICA · ENTENDER TU PROPIO PROYECTO

Login con JWT y mapa del código

Cómo viaja el token desde el login hasta cada petición, explicado con la analogía de la pulsera de hotel. Quarkus y Spring lado a lado, qué hace cada archivo y las preguntas típicas sobre seguridad.

Java 21 · Oracle · Angular 22 · JWT · Spring Boot · Quarkus

Proyecto: `login-jwt` (login con JWT, roles ADMIN y VIEW, CRUD sobre Oracle)

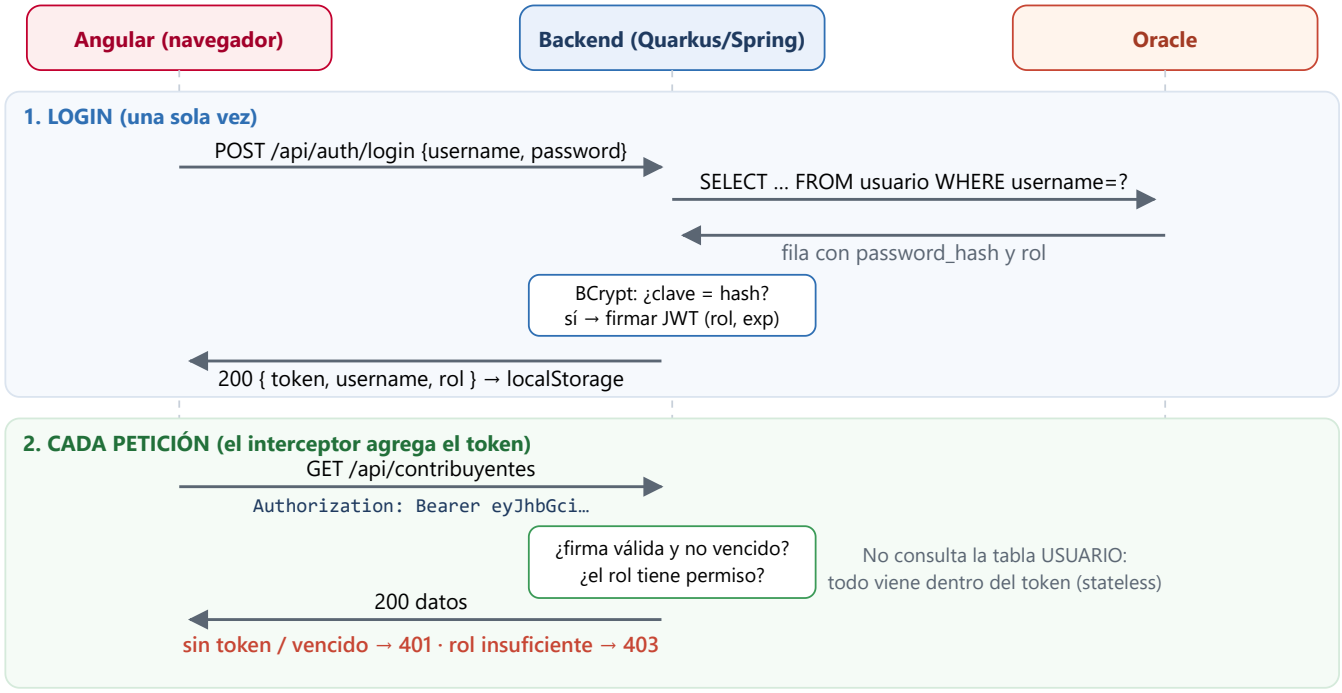
Cómo funciona el login con JWT

La analogía: la pulsera de un hotel todo incluido

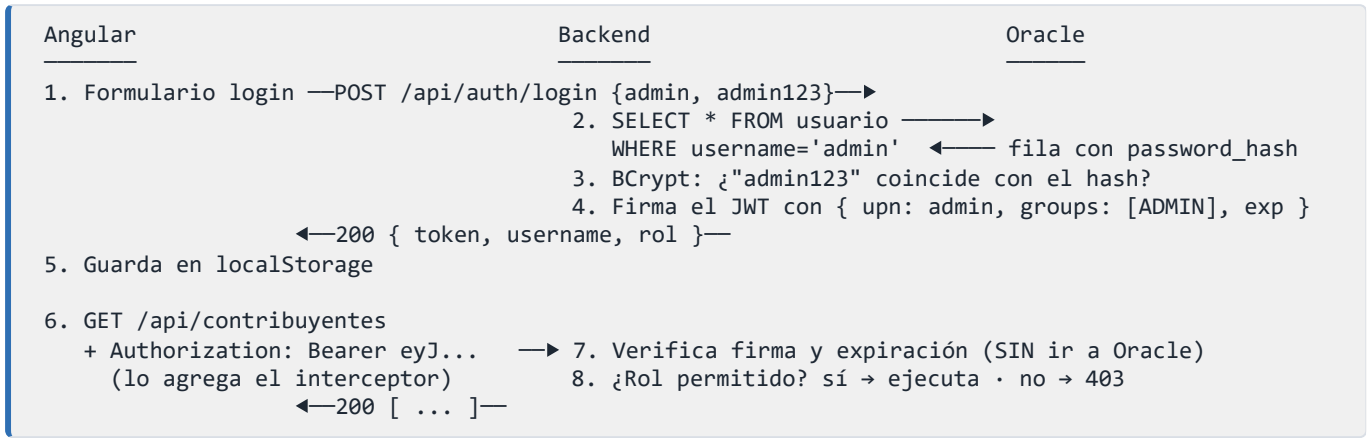
1. Llegas a **recepción** y muestras tu cédula (usuario + contraseña).
2. Recepción revisa la lista, y si estás, te pone una **pulsera**: dice tu nombre, tu tipo de plan (rol) y hasta cuándo vale (expiración). Tiene un **sello** imposible de falsificar (firma).
3. En el restaurante, en la piscina o en el bar **no vuelven a pedir tu cédula**: solo miran la pulsera.
4. ¿El sello es auténtico y no venció? → pasas.
5. ¿Tu plan incluye el bar? → si es "solo comidas", te dicen que no (**403**).
6. Sin pulsera → ni siquiera entras (**401**).

Hotel	Nuestra app
Recepción	`POST /api/auth/login`
Cédula	`{ "username", "password" }`
Pulsera	El **token JWT**
Plan "todo incluido" / "solo comidas"	Rol `ADMIN` / `VIEW`
Sello	Firma del token (RSA en Quarkus, HMAC en Spring)
Mostrar la pulsera	Header `Authorization: Bearer <token>`
El guardia del bar	`@RolesAllowed` (Quarkus) / `SecurityConfig` (Spring)

El flujo completo



El mismo flujo, paso a paso:



Lo importante del paso 7: el backend **no guarda sesiones**. Todo lo que necesita saber viene dentro del token. Por eso se dice que JWT es **stateless** (sin estado).

¿Qué tiene un JWT?

Tres partes en Base64URL separadas por puntos: `HEADER.PAYLOAD.FIRMA`

```
// HEADER
{ "alg": "RS256", "typ": "JWT" }

// PAYLOAD (los "claims")
{
  "iss": "login-jwt",          // quién lo emitió
  "upn": "admin",              // quién es
  "groups": ["ADMIN"],         // roles
  "rol": "ADMIN",
  "iat": 1794427665,           // cuándo se emitió
  "exp": 1794431265            // cuándo vence (1 hora después)
}
```

> ⚠️ El payload **no está cifrado**, solo codificado: cualquiera lo puede leer en https://jwt.io. **Nunca pongas la contraseña ni datos sensibles en el token.** La firma no lo oculta: solo garantiza que **nadie lo modificó**.

Quarkus vs Spring: lo mismo con distintas herramientas

Paso	Quarkus	Spring Boot
Extensión / starter	`smallrye-jwt` + `smallrye-jwt-build`	`security` + `oauth2-resource-server`
Comparar contraseña	`BcryptUtil.matches(clave, hash)`	`passwordEncoder.matches(clave, hash)`
Generar token	`Jwt.issuer(..).upn(..).groups(..).sign()`	`jwtEncoder.encode(JwtEncoderParameters.from(header, claims))`
Algoritmo de firma	**RS256**: firma con `privateKey.pem` y verifica con `publicKey.pem`	**HS256**: una sola clave secreta (`app.jwt.secret`)
Validar token en cada petición	Automático (`mp.jwt.verify.*` en properties)	Automático (`.oauth2ResourceServer(jwt)` en `SecurityConfig`)
Reglas por rol	`@RolesAllowed("ADMIN")` en cada método	`.requestMatchers(...).hasRole("ADMIN")` en un solo archivo
Endpoint público	`@PermitAll`	`.permitAll()`

RS256 vs HS256: con RS256 (asimétrico) solo quien tiene la clave privada puede firmar, pero cualquiera con la pública puede verificar; sirve cuando varios microservicios validan tokens. HS256 (simétrico) usa la misma clave para todo: es más simple, pero todos los que verifican pueden también firmar.

Del lado de Angular

Archivo	Qué hace	Analogía
`core/auth.service.ts`	Llama al login, guarda el token, sabe el rol (`esAdmin`)	La recepción
`core/auth.interceptor.ts`	Agrega `Authorization: Bearer` a **cada** petición; si llega 401, cierra la sesión	Un middleware de Express/Laravel, pero en el cliente
`core/auth.guard.ts`	No deja entrar a `/contribuyentes` sin sesión	El portero
`login/login.ts`	Formulario reactivo con validaciones	El formulario de registro

> **Ocultar botones en Angular NO es seguridad.** Cualquiera puede llamar a la API con Postman. La seguridad real está en el backend (403). Ocultar es solo para que la pantalla tenga sentido para cada rol.

Validaciones del login implementadas

Cliente (Angular): usuario obligatorio, de 3 a 50 caracteres y sin espacios; contraseña obligatoria, de 6 a 100 caracteres; errores visibles solo después de tocar el campo; botón deshabilitado mientras se espera; la contraseña se limpia tras un intento fallido; botón Ver/Ocultar contraseña; si ya hay sesión, `/login` redirige; si el token guardado venció, se descarta al cargar.

Servidor: `@NotBlank` y `@Size` (400); usuario inexistente, clave incorrecta o usuario inactivo responden **el mismo** 401 "Usuario o contraseña incorrectos", para no revelar qué usuarios existen; las contraseñas se guardan en **BCrypt** y nunca en texto plano; el token vence en 1 hora.

Generar el hash BCrypt de un usuario nuevo

Cualquiera de estas opciones:

```
# Python
pip install bcrypt
python -c "import bcrypt; print(bcrypt.hashpw(b'miClave', bcrypt.gensalt(10,
prefix=b'2a')).decode())"
```

```
// Java, en un main o test temporal (Spring)
System.out.println(new BCryptPasswordEncoder().encode("miClave"));
// Quarkus
System.out.println(io.quarkus.elytron.security.common.BcryptUtil.bcryptHash("miClave"));
```

```
INSERT INTO usuario (username, password_hash, rol) VALUES ('nuevo', '$2a$10$...', 'VIEW');
COMMIT;
```

Preguntas que te pueden hacer sobre esto

- **¿Dónde guardas el token y qué riesgos tiene?** En `localStorage`, por simplicidad. Si la página tiene una vulnerabilidad XSS, un script malicioso podría leerlo. La alternativa más segura es una cookie `HttpOnly` + `Secure` + `SameSite`, que JavaScript no puede leer, aunque en ese caso hay que proteger contra CSRF.
- **¿Cómo cierras sesión si el backend no guarda nada?** Borrando el token en el cliente. El token sigue siendo válido hasta que vence; por eso se usan tiempos cortos. Para revocarlo de inmediato habría que tener una lista negra o usar *refresh tokens*.
- **¿Qué es un refresh token?** Un segundo token, de vida larga, que solo sirve para pedir un nuevo access token sin volver a escribir la contraseña. Aquí no se implementó para mantener la prueba simple.
- **¿Por qué 401 y no 403 en el login fallido?** 401 significa "no sé quién eres" (no autenticado). 403 significa "sé quién eres, pero no tienes permiso" (no autorizado).
- **¿Por qué desactivar CSRF en Spring?** Porque la API no usa cookies de sesión: el token va en un header que el navegador no envía solo. CSRF ataca justamente a las cookies que se mandan automáticamente.

Mapa de archivos: qué hace cada uno

Los dos backends tienen la **misma forma**: un paquete `auth` (login) y otro `contribuyente` (CRUD).

backend-quarkus (8081)

```
src/main/resources/
  application.properties   Oracle, JWT, CORS, Swagger           ← datos de conexión aquí
  privateKey.pem           Firma los tokens (solo el backend)
  publicKey.pem            Verifica los tokens
src/main/java/com/hacienda/
  auth/
    Usuario.java           @Entity tabla USUARIO + porUsername()
    LoginRequest.java      DTO {username, password} con @NotBlank
    LoginResponse.java     DTO {token, username, rol, expiraEnSegundos}
    AuthResource.java      POST /login (BCrypt + Jwt.sign) · GET /me
  contribuyente/
    Contribuyente.java     @Entity tabla CONTRIBUYENTE + existeNit()
    ContribuyenteRequest   DTO del POST/PUT con validaciones
    ContribuyenteResource  CRUD + @RolesAllowed por método           ← permisos aquí
```

Patrón: **Panache Active Record**. La entidad trae `findById`, `listAll`, `persist` y `deleteById`, como un Model de Eloquent. No hay Repository ni Service, para no agregar capas en la prueba.

backend-spring (8080)

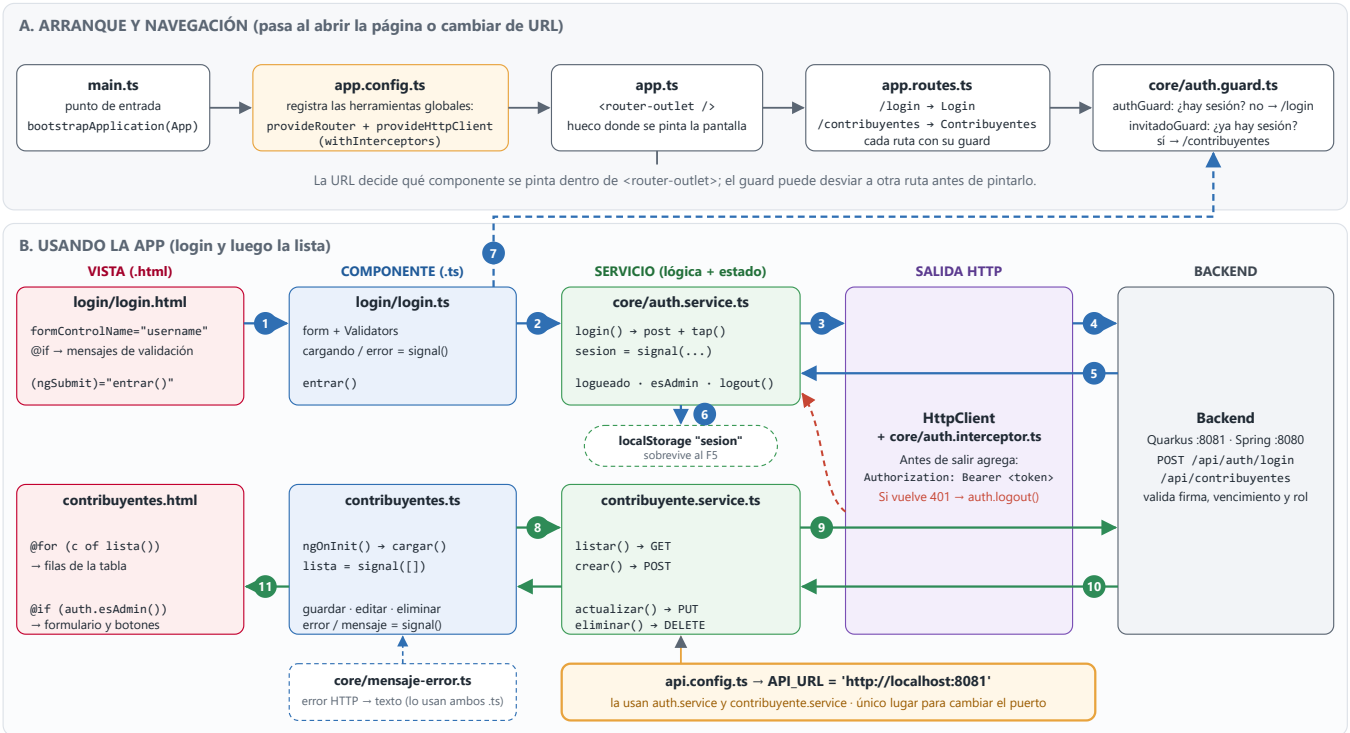
```
src/main/resources/
  application.properties   Oracle, clave JWT, CORS                ← datos de conexión aquí
src/main/java/com/hacienda/
  BackendSpringApplication.java   main()
  config/
    SecurityConfig.java           TODA la seguridad: rutas públicas, roles, JWT, CORS, BCrypt  ←
permisos aquí
    GlobalExceptionHandler.java   Errores de @Valid -> 400 con la lista de campos
  auth/
    Usuario.java                  @Entity tabla USUARIO
    UsuarioRepository.java        findByUsername (Spring Data arma el SQL por el nombre)
    LoginRequest / LoginResponse  DTOs
    AuthController.java           POST /login (BCrypt + JwtEncoder) · GET /me
  contribuyente/
    Contribuyente.java            @Entity tabla CONTRIBUYENTE
    ContribuyenteRepository.java  JpaRepository + existsByNit
    ContribuyenteRequest.java     DTO con validaciones
    ContribuyenteController.java  CRUD (sin reglas de rol: están en SecurityConfig)
```

Patrón: **Controller → Repository** (Spring Data JPA). En un proyecto grande iría un `@Service` en medio, con las reglas de negocio.

La misma cosa en los dos

Concepto	Quarkus	Spring
Endpoint REST	`@Path` + `@GET`	`@RestController` + `@GetMapping`
Parámetro de URL	`@PathParam`	`@PathVariable`
Body	parámetro directo	`@RequestBody`
Inyección	`@Inject`	constructor (o `@Autowired`)
Validar body	`@Valid`	`@Valid`
Transacción	`jakarta.transaction.Transactional`	`org.springframework...Transactional`
Respuesta con código	`Response.status(409).entity(..)`	`ResponseEntity.status(409).body(..)`
Consultas	métodos estáticos en la entidad (Panache)	interfaz `JpaRepository`

frontend (Angular 22, 4200)



LOGIN

1 El botón Ingresar dispara (ngSubmit) → entrar() en login.ts
2 login.ts valida el formulario y llama a auth.login(usuario, clave)
3 auth.service hace http.post(API_URL + '/api/auth/login')
4 El interceptor la deja pasar sin token (todavía no hay) → backend
5 El backend responde 200 { token, username, rol }
6 tap() guarda la sesión en localStorage y en el signal sesion
7 login.ts navega a /contribuyentes → pasa por app.routes y authGuard (A)

LISTA Y CRUD

8 Al abrir la pantalla, ngOnInit() → cargar() → servicio.listar()
9 http.get(...) → el interceptor agrega Authorization: Bearer <token>
10 El backend valida el token y el rol → 200 [...] (o 403 si no puede)
11 lista.set(datos) → el @for del .html se vuelve a pintar solo

- - - 401 en cualquier petición:
el interceptor llama a auth.logout() → borra la sesión → /login

Crear, editar y eliminar siguen el mismo camino que 8 a 11 (POST, PUT, DELETE).

```
src/app/
  api.config.ts                 URL del backend                    ← CAMBIAR AQUÍ 8081 / 8080
  app.config.ts                 provideHttpClient + interceptor
  app.routes.ts                 /login y /contribuyentes (con guards)
  core/
    auth.service.ts             login, logout, token, esAdmin (signals)
    auth.interceptor.ts         agrega Bearer; 401 => logout
    auth.guard.ts               authGuard (privado) · invitadoGuard (login)
    mensaje-error.ts            entiende los errores de los dos backends
  login/
    login.ts / login.html       formulario reactivo + validaciones
  contribuyentes/
    contribuyente.service.ts    llamadas HTTP del CRUD
    contribuyentes.ts / .html   tabla (todos) + formulario y botones (solo ADMIN)
```

> Angular 22 es **zoneless**: la pantalla se actualiza cuando cambia un `signal()`. Por eso el estado (`lista`, `error`, `cargando`) está en signals. Si asignas una variable normal dentro de un `subscribe`, la vista **no** se refresca.